KeroshiZeroTetris:
Self-Learning Modern Tetris AI

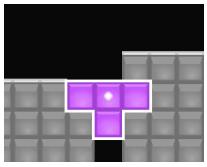
A Preprint

KevinLikesCoding
Shanghai Caoyang NO.2 High School
Shanghai, China
kevinlikescoding@gmail.com

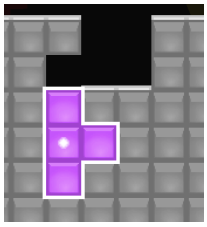
May 2026

1 Introduction

Traditional Tetris is a classic single-player survival game, and Modern Tetris[1] introduces something new. Modern Tetris is usually played in a 1v1 competitive mode, and features an attack system. Players can perform specific clears to send garbage lines to their opponents. In addition, Modern Tetris also introduces the Super Rotation System (SRS), so players can perform clears like T-Spin to gain more attack.



(a) A T-Spin Double.



(b) A T-Spin Triple.

Figure 1: Some sample clears with SRS.

In Modern Tetris, human players often try to perform higher Attack Per Minute (APM), but Tetris AI needs to perform higher Attack Per Piece (APP).

To achieve a higher APP, we present KeroshiZeroTetris based on AlphaZero[1].

However, directly applying the AlphaZero framework to Modern Tetris brings some difficulties:

- Unlike chess or Go, the next state is not fixed because of the 7-Bag random generator.
- The neural network for Modern Tetris must be carefully designed to predict both policy and value information.
- The Tetris AI needs to play indefinitely with a high APP, so a major challenge is how to set up training tasks and how to implement the trained policy in actual match play.

To solve these issues, this paper details the implementation of KeroshiZeroTetris.

[1] Examples include competitive platforms like TETR.IO and Techmino, as well as official games like Tetris Effect: Connected.

2 Model

A Tetris model requires the following inputs:

- The board.
- The piece types of the current, hold and next queue.
- The values including Combo and Back-to-Back (B2B).

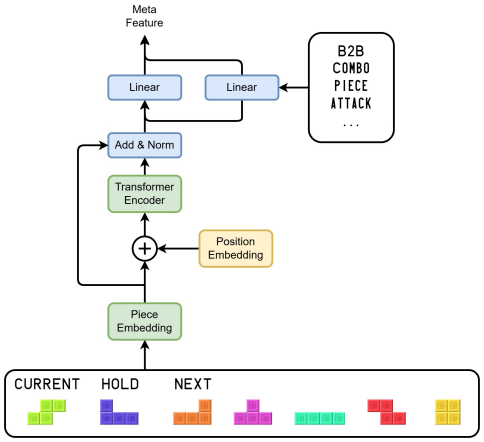


Figure 2: The piece sequence encoder.

We call the piece types the piece sequence. First we need to encode this sequence.
First, we put the piece sequence into a Piece Embedding layer. Then we add the Position Embedding to it. After that, we send it into the Transformer Encoder[2]. To make the model converge faster, we bring the original piece sequence embedding to do a residual connection[3]. Now, the piece sequence processing is finished.

Next, we need to add the game data. We use a Linear layer on the game data first. Second, we join it with the sequence features and put them through another Linear layer. Finally, we join it with the processed game data from the first step. Now, the piece sequence and the game data are combined into a feature called Meta Feature.

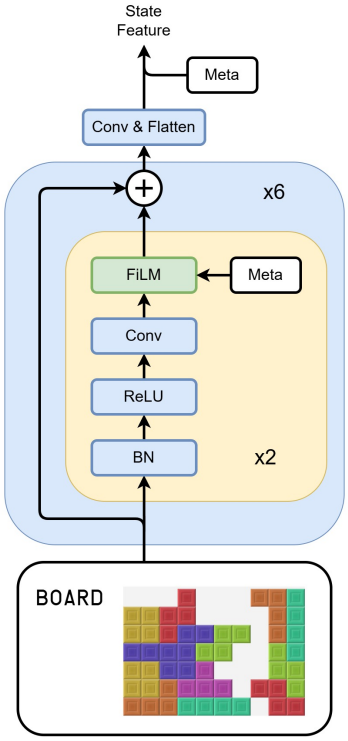


Figure 3: The board encoder.

After we process the piece sequence and game data, we need to mix them into the board. To do this, we use the FiLM[4], and use the Meta feature as the condition. In the CNN part for the board, we use residual connections[3] and pre-activation[5]. Finally we use a Convolution layer and a Flatten layer. Then we combine the output with the original Meta Feature to get the final State Feature.
Just like AlphaZero[1], we use some Linear layers to connect the State Feature to Value Head and Policy Head.

These are the detailed parameters of our model.

Component	Size / Number
Board Input	30×10
Piece Sequence Length	10
Game Values	10
Piece Embedding Dimension	16
Transformer Encoder Layers	1
Meta Feature Dimension	256
CNN Channels	64
Residual Blocks	6
Flattened Board Feature	1200
State Feature Dimension	1456
Value Head	$1456 \rightarrow 256 \rightarrow 1$
Policy Head	$1456 \rightarrow 512 \rightarrow 1537$

References

[1] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. Science, 362(6419):1140–1144, 2018.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.

[4] Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In Proceedings of the AAAI conference on artificial intelligence, volume 32, 2018.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In European conference on computer vision, pages 630–645. Springer, 2016.